Program and Branch Name: *B.Tech CSE (AI)*

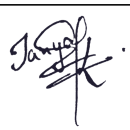
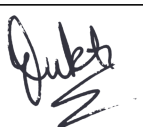
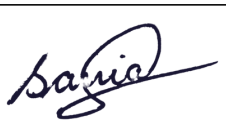
PBL Progress Report – Final

Field	Details
Course Name	Object Orient Programming Using Java
Course Code	CCSE0355
Faculty Name	Dr Sarika Agarwal
Assignment Type	Group Assignment
Student(s) Name	Tanya Kasaudhan, Yukti Dwivedi, Sachi Srivastava, Sania Tasneem, Shubham Mishra
ERP ID (I Year) / Roll No. (II Year onwards)	2501331520209, 2501331520228, 2501331520169, 2501331520174, 2501331520202
Team/Group No. *(Group/Capstone only)*	48
Problem Statement / Project Title	Student Placement Management System
SDG(s)	SDG 4: Quality Education (primary); SDG 8 (supporting)
Reporting Month	Month 3 – Final
Date of Submission	24/10/2026

Student Declaration

I/We certify that this assignment is my/our own work, with all sources appropriately acknowledged. I/We understand that a 0% mark and further academic penalties may apply if the content is found to be plagiarised or inappropriately AI-generated.

Student Name & Signature(s): _____
Date: _____24/10/2026_____

				
Tanya Kasaudhan	Yukti Dwivedi	Sachi Srivastava	Sania Tasneem	Shubham Mishra

2. Student Details / Team Details

S.No.	Student Name	Roll No.	Role / Contribution
1	Tanya Kasaudhan	2501331520209	Team leader; requirements, planning, report writing
2	Yukti Dwivedi	2501331520228	Database and testing; record handling, test plan
3	Sachi Srivastava	2501331520169	Frontend; Swing screens, tables, forms, dashboard
4	Sania Tasneem	2501331520174	Backend; model classes, services, validation
5	Shubham Mishra	2501331520202	DSA; search, sort, queue, stack, eligibility

3. Executive Summary

Campus placement records are usually scattered across spreadsheets, registers and chat groups, which causes duplicate entry and slow searching. This project delivers a Java Swing desktop application that keeps students, companies, drives, applications and placements in one place. It offers login, a dashboard and table-based screens with validation, search and sort. The result is a working prototype built with object-oriented principles; data is held in memory, and a database is planned as future work.

4. Introduction

A placement cell invites companies, shares job details, collects applications, runs selection rounds and records outcomes. The course Object Oriented Programming Using Java asks students to apply classes, inheritance, polymorphism, collections, exception handling and GUI programming to a real problem, which placement management suits well. The objectives are to design classes from real entities, build a Swing interface, handle events, process data with collections and algorithms, and test and document the result. The scope covers administrator login, five record types, search, sort and validation. Permanent storage, student portals, notifications and document upload are outside this version.

5. Problem Definition

Problem statement: to design and implement a Java Swing application that brings together the basic records needed to manage student placement activities and makes them easy to add, view, search and update. When handled manually, student, company and application data is hard to track, so a centralised system is needed.

Objectives: provide a clear interface to every module; store student and company data in structured tables; link applications to drives; record placement outcomes; apply OOP, event handling and collections; and validate input while stating limitations honestly. The target user is the placement officer, while students, companies and evaluators are represented as records or reviewers. Constraints are local single-computer use, in-memory storage, standard-library Swing only and the semester timeline. Success means easy navigation, working create-read-update-delete operations, helpful error messages and a demonstrable end-to-end workflow. Linked goals are SDG 4, 8 and 9.

6. Industry / Application Overview

A typical placement cycle runs through company outreach, eligibility definition, student registration, application, selection rounds, outcome and reporting. Colleges commonly rely on spreadsheets, which are free and familiar but create many copies and hold no relationships

between records. Email and chat groups communicate quickly but lose information. Dedicated placement platforms offer accounts, notifications and analytics, yet need cost, setup and administration effort. Our review of such systems showed that they share five core entities: student, company, drive, application and placement.

The gap addressed here is a simple, local tool that shows how related records connect and how eligibility can be checked automatically. It does not replace a secure institutional platform, as it lacks persistent storage, multi-user access, audit history and verified security.

Tool	Strength	Weakness
Spreadsheets	Free, familiar, flexible	Many copies, no relationships
Email / chat groups	Fast communication	Information lost, status hard to track
Placement platforms	Accounts, notifications, analytics	Cost and administration effort
This prototype	Simple, local, shows relationships	No persistent storage, single user

Table 1: Comparison of commonly used tools

7. Module 1: Login and Dashboard

Concepts applied: Swing components, event handling and layout managers. The login window uses JFrame, JLabel, JTextField, JPasswordField and JButton arranged with GridBagLayout. On pressing Login, an ActionListener compares the entered values with the demonstration credential; a match closes the window and opens the dashboard, otherwise an error dialog appears. The dashboard shows summary cards for total students, companies, drives, applications and placements, each reading the size of its list so counts update as records change. Navigation buttons open each module, and every module has a Back button, so the layout stays familiar. Finding: separating navigation from data operations made later screens easy to add. The fixed credential is for classroom use only; a real system needs hashed passwords.

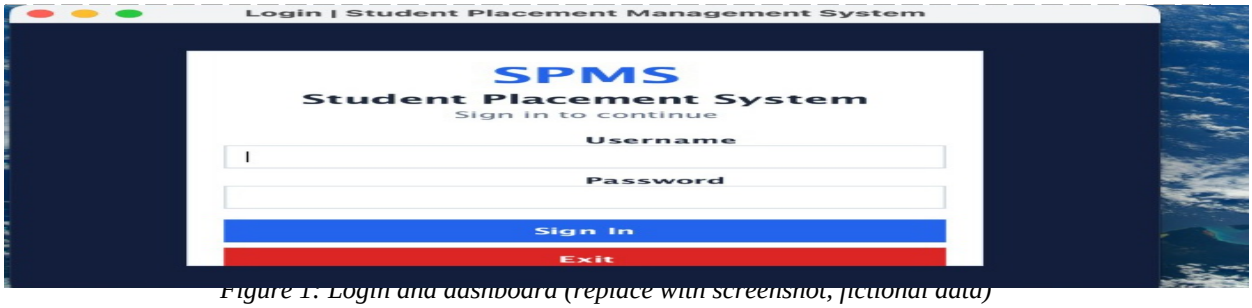


Figure 1: Login and dashboard (replace with screenshot, fictional data)

8. Module 2: Student and Company Management

Concepts applied: data representation, collections, table models, selection handling, validation and CRUD operations. Each student and company is an object stored in an ArrayList and displayed through a JTable with DefaultTableModel. Create validates the form, rejects a duplicate ID and adds the object and its row. Update locks the ID and refreshes both object and row. Delete asks for confirmation. Search filters rows by keyword. The table is always rebuilt from the list, so screen and data cannot disagree. Common cases such as no selected row, empty fields or an empty table show a clear message instead of failing. The company screen reuses the same pattern with a warning before deleting a company that has drives.

Screen	Key columns	Rules
Students	ID, name, branch, CGPA, backlogs, email, phone, placed	Unique ID; CGPA 0 to 10; 10-digit phone
Companies	ID, name, industry, contact, email, location	Unique ID; name required

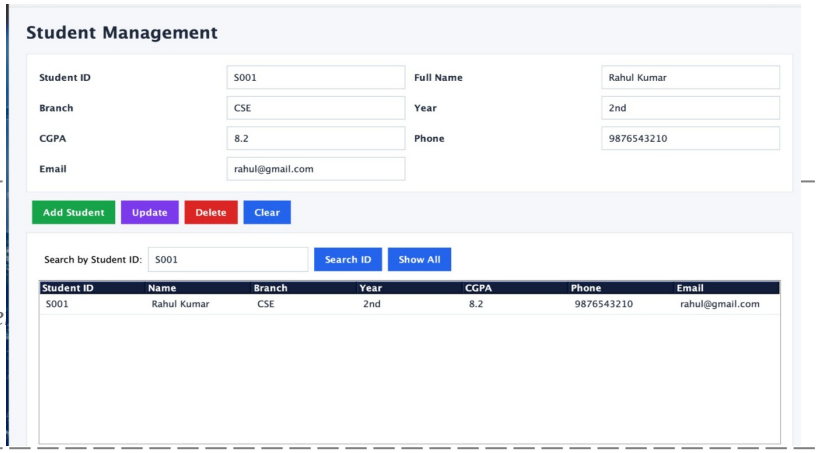


Figure 2: Student management screen (replace with screenshot)

9. Module 3: Placement Drive Management

Concepts applied: structured record management, event-driven forms and linking an opportunity to its company and requirements. The administrator picks a company from a JComboBox filled from the company list, then enters the role, minimum CGPA, maximum backlogs, allowed branches, package in LPA and drive date. The eligibility rules are stored with the drive as explicit values and later used by the application module; no assumption is made from missing data.

Validation ensures that a company is chosen, CGPA is a number from 0 to 10, backlogs are a whole number of zero or more, the package is positive and the date follows dd/MM/yyyy. When a value is missing or wrong, the interface shows an understandable message and keeps the form open for correction.

Work completed includes the drive table, the add and edit form, the drop-down linked to companies and the validation checks. Findings: linking each drive to exactly one company through its company ID kept the one-to-many relationship simple, and storing eligibility rules with the drive avoided repeated checks elsewhere.

Column	Meaning
Drive ID, Company, Role	Unique identifier, employer name and job or internship title
Min CGPA, Max Backlogs, Branches	Eligibility criteria stored with the drive
Package (LPA), Date	Offered package and drive date

Table 3: Drive table columns



Figure 3: Placement drive screen (replace with screenshot)

10. Module 4: Application Management

Concepts applied: relationships between records, status tracking, selection events and validation. The administrator chooses a student and a drive. The program checks eligibility using three explicit rules: the student's CGPA must be at least the drive minimum, backlogs must not exceed the drive maximum, and the student's branch must be in the allowed list. If all hold, an application is created with the status Applied; otherwise it is refused with the reason stated. As rounds finish, the status changes to Shortlisted, Selected or Rejected.

Further rules prevent a student from applying twice to the same drive, and each row shows which student and drive are linked. Because the prototype holds data in lists, the program performs these checks in code; in a database version, foreign-key constraints could enforce them automatically. Findings: the eligibility rule is applied the same way every time, which removes the manual comparison of numbers that placement officers otherwise do by eye.

Work completed includes the application table, student and drive selectors, the eligibility refusal message, duplicate-application check and the status-change control.

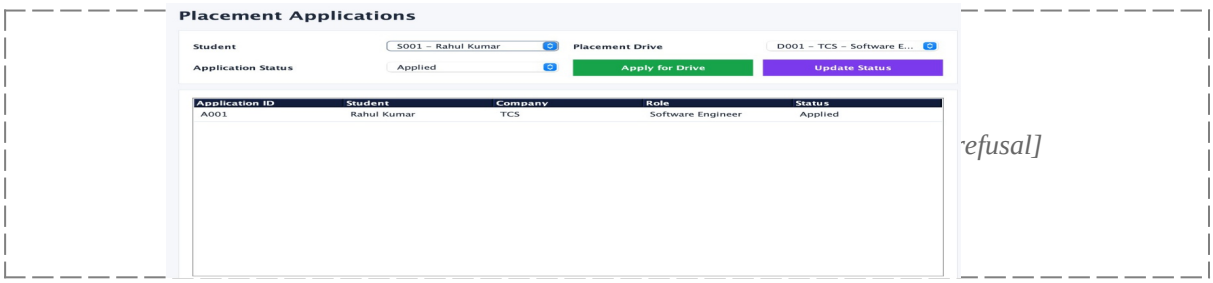


Figure 4: Application screen (replace with screenshot)

11. Module 5: Placement Records

Concepts applied: outcome records, table-based presentation and linking a final placement to student and employer. When an application has the status Selected, the administrator can create a placement record containing placement ID, student, company, role, package and date. The student's Placed field is then updated and the dashboard count increases.

The screen also calculates total placed students, highest package and average package from the list each time it opens, so the figures always agree with the data. The quality of these summaries depends on accurate input in earlier modules, because wrong data upstream produces wrong results here.

Work completed includes the placement table, the creation flow from a selected application and the summary figures. Findings: deriving summaries from the lists, instead of storing them separately, prevented the numbers from drifting out of step. This module completes the chain from student and company through drive and application to placement, and it is what makes the end-to-end demonstration possible.



Figure 5: Placement records screen (replace with screenshot)

12. Final Integrated Solution / Testing and Validation

The five modules combine into a single workflow. The administrator logs in, prepares student and company records, creates a drive for a company, records applications that link students to the drive, updates statuses as selection proceeds and enters selected students as placements; the dashboard counts reflect each change. The program follows a layered structure: presentation, event, service, data and model layers, so rules can be changed without touching the interface and the in-memory data layer can later be replaced by a database.

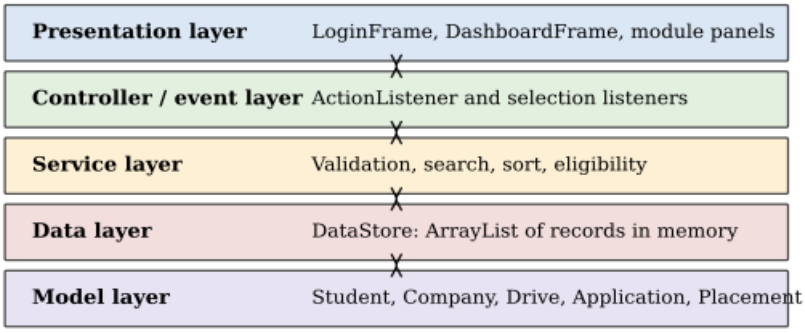


Figure 6: Layered architecture

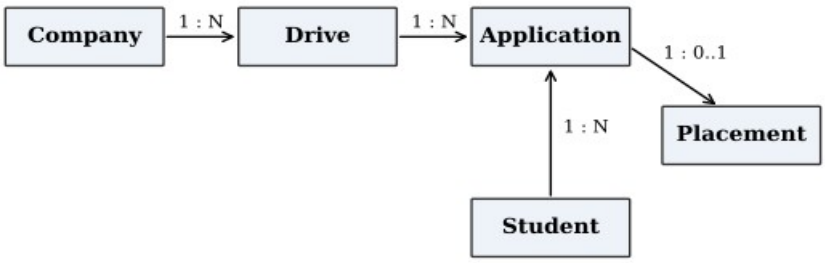


Figure 7: Entity relationships

Testing followed three steps: module-level checks with valid and invalid input, integration checks through the whole chain and a user-style check by someone who did not write the code. Validation covers empty fields, duplicate IDs, non-numeric values, out-of-range CGPA, missing selection and ineligible applications. The detailed test plan is in Appendix D. Results are recorded only after each test is actually run on the final build; they are entered by the team in the Result column and summarised there, together with screenshots as evidence.

13. Expected Benefits / Learning Outcomes

For the placement cell, one interface replaces several unconnected sheets, linked records reduce repeated typing, eligibility rules are applied consistently and counts are available immediately. For students and the institution, better organised placement activity supports the path from education to employment, which relates to SDG 4, 8 and 9.

The team gained practical command of the course outcomes: classes and objects, encapsulation, inheritance, polymorphism and interfaces, exception handling, collections, Swing GUI programming and event handling. We also practised algorithms such as linear and binary search, sorting, queues and stacks, and the software engineering habits of writing requirements, designing, testing and documenting. A further lesson was that recording limitations honestly is part of good engineering.

Course concept	How the project used it
Encapsulation	Private fields with getters and setters; ID has no setter
Inheritance / polymorphism	Shared base fields; overriding, overloading and interfaces
Exception handling	try-catch for bad numbers and missing selection
Collections and GUI	ArrayList, HashSet; JFrame, JTable, dialogs, listeners

Table 4: Course concepts applied

14. Challenges Faced

The first challenge was scope, because placement systems can grow very large; we limited the work to five record types and a simple login. Keeping tables and data consistent was another: edits or deletes could leave the table out of date, so we rebuilt the table from the list after every change. Users could press Edit or Delete without selecting a row, which we handled with a selection check and a message. Bad data confused later records, so we collected checks in a reusable Validator class.

Time and teamwork also mattered. Five members worked on different parts, so we agreed class names and interfaces early and merged work step by step. Data being lost on exit is a known limit that we documented and planned to remove with a database. Testing was manual, which is slower and less thorough than automated testing; adding JUnit tests is a planned improvement.

15. Future Improvements

The most important next step is persistent storage using JDBC with SQLite or MySQL, followed by secure login with password hashing and role-based access for administrators, students and companies. Further additions are CSV and PDF export, charts by branch and package, email notifications, resume upload with access control, audit logs, backups, automated JUnit tests and a multi-user web version with a Spring Boot backend.

16. Conclusion

The Student Placement Management System shows how a Java Swing application can organise the core records of a placement cell. Login and dashboard lead to student, company, drive, application and placement modules, using GUI design, event-driven programming, collections and basic algorithms in a realistic setting. The in-memory design suits demonstration but not dependable institutional use, since records vanish on exit and security is limited. Persistent storage, stronger authentication and systematic testing are the key next steps. The project also gave the team practice in planning, dividing tasks, validating input, testing and documenting.

17. References

[1] Oracle, "The Java Tutorials: Creating a GUI With Swing." https://docs.oracle.com/javase/tutorial/uiswing/ (accessed Oct. 2026).

[2] Oracle, "Java Platform, Standard Edition Documentation." https://docs.oracle.com/en/java/ (accessed Oct. 2026).

[3] Oracle, "Collections Framework Overview." https://docs.oracle.com/javase/8/docs/technotes/guides/collections/ (accessed Oct. 2026).

[4] Oracle, "JTable," Java Platform API Documentation. https://docs.oracle.com/javase/8/docs/api/javax/swing/JTable.html (accessed Oct. 2026).

[5] Oracle, "ArrayList," Java Platform API Documentation. https://docs.oracle.com/javase/8/docs/api/java/util/ArrayList.html (accessed Oct. 2026).

[6] Oracle, "Concurrency in Swing: The Event Dispatch Thread." https://docs.oracle.com/javase/tutorial/uiswing/concurrency/dispatch.html (accessed Oct. 2026).

[7] Oracle, "JDBC Basics," The Java Tutorials. https://docs.oracle.com/javase/tutorial/jdbc/basics/ (accessed Oct. 2026).

[8] United Nations, "The 17 Goals: Sustainable Development Goals." https://sdgs.un.org/goals (accessed Oct. 2026).

18. Appendix A – Source Code

Well-commented excerpts of the main classes are given below. The complete source and repository link are submitted separately.

A.1 Main.java – start-up on the Event Dispatch Thread

```
import javax.swing.SwingUtilities;

public class Main {
    public static void main(String[] args) {
        // Swing screens must be created on the EDT
        SwingUtilities.invokeLater(() -> new LoginFrame().setVisible(true));
    }
}
```

A.2 Student.java – encapsulated model class

```
public class Student {
    private final String studentId;   // no setter: ID never changes
    private String name, branch;
    private double cgpa;
    private int backlogs;
    private boolean placed;

    public Student(String studentId, String name, String branch,
                   double cgpa, int backlogs) {
        this.studentId = studentId; this.name = name;
        this.branch = branch; this.cgpa = cgpa; this.backlogs = backlogs;
    }
    public String getStudentId() { return studentId; }
    public String getName()      { return name; }
    public String getBranch()    { return branch; }
    public double getCgpa()      { return cgpa; }
    public int getBacklogs()     { return backlogs; }
    public boolean isPlaced()    { return placed; }
    public void setName(String n)     { this.name = n; }
    public void setBranch(String b)   { this.branch = b; }
    public void setCgpa(double c)     { this.cgpa = c; }
    public void setBacklogs(int b)    { this.backlogs = b; }
    public void setPlaced(boolean p)  { this.placed = p; }
}
```

A.3 DataStore.java – in-memory storage with duplicate check

```
import java.util.ArrayList;
import java.util.List;

public class DataStore {
    private final List<Student> students = new ArrayList<>();
    private final List<Company> companies = new ArrayList<>();

    public boolean addStudent(Student s) {
        for (Student existing : students) {
            if (existing.getStudentId().equalsIgnoreCase(s.getStudentId())) {
                return false;             // duplicate ID refused
            }
        }
        students.add(s);
        return true;
    }
    public List<Student> getStudents() {
        return new ArrayList<>(students); // copy protects the original
    }
```

```
    public boolean removeStudent(String id) {
        return students.removeIf(s -> s.getStudentId().equals(id));
    }
}
```

A.4 Login check (classroom demonstration only)

```
loginButton.addActionListener(e -> {
    String user = userField.getText().trim();
    String pass = new String(passField.getPassword());
    if (user.equals("admin") && pass.equals("admin")) {
        dispose();
        new DashboardFrame().setVisible(true);
    } else {
        JOptionPane.showMessageDialog(this, "Invalid username or password.",
            "Login failed", JOptionPane.ERROR_MESSAGE);
    }
});
```

A.5 Refreshing a JTable and deleting with a selection check

```
private void refreshTable(List<Student> list) {
    model.setRowCount(0);
    for (Student s : list) {
        model.addRow(new Object[]{ s.getStudentId(), s.getName(),
            s.getBranch(), s.getCgpa(), s.getBacklogs() });
    }
}

deleteButton.addActionListener(e -> {
    int row = table.getSelectedRow();
    if (row == -1) {
        JOptionPane.showMessageDialog(this, "Please select a row first.");
        return;
    }
    String id = (String) model.getValueAt(row, 0);
    int choice = JOptionPane.showConfirmDialog(this,
        "Delete student " + id + "?", "Confirm", JOptionPane.YES_NO_OPTION);
    if (choice == JOptionPane.YES_OPTION) {
        store.removeStudent(id);
        refreshTable(store.getStudents());
    }
});
```

A.6 Search, sort and eligibility

```
public List<Student> searchByName(String keyword) {
    List<Student> result = new ArrayList<>();
    String k = keyword.toLowerCase();
    for (Student s : students) {                // linear search, O(n)
        if (s.getName().toLowerCase().contains(k)) result.add(s);
    }
    return result;
}

// Sort by CGPA, highest first
List<Student> list = store.getStudents();
Collections.sort(list, (a, b) -> Double.compare(b.getCgpa(), a.getCgpa()));
refreshTable(list);

public static boolean isEligible(Student s, Drive d) {
    boolean cgpaOk    = s.getCgpa() >= d.getMinCgpa();
    boolean backlogOk = s.getBacklogs() <= d.getMaxBacklogs();
    boolean branchOk  = d.getAllowedBranches().toLowerCase()
```

```
                        .contains(s.getBranch().toLowerCase());
    return cgpaOk && backlogOk && branchOk;
}
```

A.7 Validation with exception handling

```
try {
    double cgpa = Double.parseDouble(cgpaField.getText().trim());
    if (cgpa < 0 || cgpa > 10) {
        throw new IllegalArgumentException("CGPA must be between 0 and 10.");
    }
    // continue saving
} catch (NumberFormatException ex) {
    JOptionPane.showMessageDialog(this, "CGPA must be a number.");
} catch (IllegalArgumentException ex) {
    JOptionPane.showMessageDialog(this, ex.getMessage());
}
```

A.8 Binary search by ID and compile commands

```
int binarySearch(List<Student> sorted, String key) {
    int low = 0, high = sorted.size() - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        int cmp = sorted.get(mid).getStudentId().compareTo(key);
        if (cmp == 0) return mid;
        else if (cmp < 0) low = mid + 1;
        else high = mid - 1;
    }
    return -1;                                    // O(log n)
}

// Compile and run
// javac *.java
// java Main
```

A.9 Validator.java – reusable input checks

```
public class Validator {
    public static boolean isNotEmpty(String s) {
        return s != null && !s.trim().isEmpty();
    }
    public static boolean isValidEmail(String s) {
        int at = s.indexOf('@');
        return at > 0 && s.indexOf('.', at) > at + 1;
    }
    public static boolean isNumberInRange(String s, double lo, double hi) {
        try {
            double v = Double.parseDouble(s.trim());
            return v >= lo && v <= hi;
        } catch (NumberFormatException e) {
            return false;
        }
    }
    public static boolean isValidPhone(String s) {
        return s != null && s.matches("\d{10}");
    }
    public static boolean isValidDate(String s) {
        return s != null && s.matches("\d{2}/\d{2}/\d{4}");
    }
}
```

A.10 Drive.java and Application.java – associations

```
public class Drive {
```

```
    private final String driveId;
    private final Company company;        // has-a: drive belongs to a company
    private String role, allowedBranches;
    private double minCgpa, packageLpa;
    private int maxBacklogs;

    public Drive(String id, Company company, String role,
                 double minCgpa, int maxBacklogs, String branches) {
        this.driveId = id; this.company = company; this.role = role;
        this.minCgpa = minCgpa; this.maxBacklogs = maxBacklogs;
        this.allowedBranches = branches;
    }
    public double getMinCgpa()          { return minCgpa; }
    public int getMaxBacklogs()         { return maxBacklogs; }
    public String getAllowedBranches()  { return allowedBranches; }
    @Override public String toString()  { return company.getName() + " - " + role; }
}

public class Application {
    private final String applicationId;
    private final Student student;        // has-a: links student and drive
    private final Drive drive;
    private String status = "Applied";    // Applied, Shortlisted, Selected, Rejected
    public Application(String id, Student s, Drive d) {
        this.applicationId = id; this.student = s; this.drive = d;
    }
    public String getStatus()           { return status; }
    public void setStatus(String s)     { this.status = s; }
}
```

A.11 Application service – eligibility and duplicate check

```
public String apply(Student s, Drive d) {
    for (Application a : applications) {
        if (a.getStudent() == s && a.getDrive() == d) {
            return "Student has already applied to this drive.";
        }
    }
    if (s.getCgpa() < d.getMinCgpa())
        return "CGPA is below the minimum of " + d.getMinCgpa() + ".";
    if (s.getBacklogs() > d.getMaxBacklogs())
        return "Backlogs exceed the maximum of " + d.getMaxBacklogs() + ".";
    if (!d.getAllowedBranches().toLowerCase()
          .contains(s.getBranch().toLowerCase()))
        return "Branch is not eligible for this drive.";
    applications.add(new Application(nextId(), s, d));
    return null;                          // null means success
}

public void markSelected(Application a, Placement p) {
    a.setStatus("Selected");
    a.getStudent().setPlaced(true);       // dashboard count increases
    placements.add(p);
}
```

A.12 Queue and stack – review order and undo

```
import java.util.ArrayDeque;
import java.util.Deque;
import java.util.Queue;

Queue<Application> reviewQueue = new ArrayDeque<>();
reviewQueue.offer(newApplication);        // added in arrival order
```

```
Application next = reviewQueue.poll();    // oldest reviewed first

Deque<Student> undoStack = new ArrayDeque<>();
undoStack.push(deletedStudent);           // push on delete

undoButton.addActionListener(e -> {
    if (undoStack.isEmpty()) {
        JOptionPane.showMessageDialog(this, "Nothing to undo.");
        return;
    }
    store.addStudent(undoStack.pop());    // restore last deleted record
    refreshTable(store.getStudents());
});
```